
e-Sewa × WWF Hackathon 2026

Challenge 11: Intelligent Payment Scheduling & Recurring Payment Automation

Solution Documentation

Submitted by:

Team 0XNULL

Table of Contents

1	Team Introduction	2
1.1	Team Overview	3
1.2	Team Members & Responsibilities	3
1.2.1	Sansar Chhetri - Python (FastAPI)	3
1.2.2	Nirmal Bista - Database (PostgreSQL).....	4
1.2.3	Rajkumar Tiruwa – UI/UX (Figma)	4
1.2.4	Paras Adhikari – Frontend Developer (React JS)	4
1.2.5	Abishek Kunwar - Backend (NestJS)	5
1.3	Why Our Team Can Build This Solution	5
2	Problem Understanding	5
3	Proposed Solution — PaySmart.....	7
3.1	Features of the proposed solution:	8
3.1.1	Smart Bill Alerts	8
3.1.2	AI-Based Pattern Detection and Smart Suggestions.....	10
3.1.3	Flexible Scheduling System.....	13
3.1.4	Minimal-Setup Manual Scheduling	13
3.1.5	Low Balance Awareness and Preventive Alerts	14
4	In-Scope and Out-of-Scope.....	14
4.1	In-Scope	14

	3
4.1 Out of Scope	15
5 User Journey	16
6 Technology Stack.....	16
7 Architecture Overview	17
8 Expected Impact.....	17
9 Intelligent Payment Scheduling & Automation (Working Mechanism).....	17
9.1 Transaction Data Synchronization	18
9.2 Behavioral Pattern Analysis	18
9.3 Intelligent Suggestion Engine	18
9.4 User Confirmation & Auto-Setup	18
9.5 Scheduling & Pre-Execution Validation	18
9.6 Automated Payment Execution.....	19
9.7 Real-Time Notification System	19
9.8 End-to-End Flow	19
10 Conclusion	19

1 Team Introduction

We are Team 0xNull, a group of undergraduate BSc IT students from London School of Economics and Business (LBEF), united by a shared passion for building practical and user-centric digital solutions. The name 0xNull reflects our approach to identifying gaps in existing systems and transforming them into meaningful, functional experiences through technology. Although we are at an early stage in our academic journey, we bring a strong learning mindset, adaptability, and hands-on experience with modern tools such as React.js, Nest JS, PostgreSQL, and Python-based frameworks. By combining our skills in frontend development, backend systems, and intelligent logic design, we aim to develop solutions that are not only technically sound but also simple, intuitive, and impactful for real-world users.

Team Members:

Sansar Chhetri

Nirmal Bista

Rajkumar Tiruwa

Paras Adhikari

Abishek Kunwar



1.1 Team Overview

We are a team of undergraduate BSc IT students from Lord Buddha Education Foundation, currently in our second semester, united by a strong passion for building practical, user-centric digital solutions that address real-world challenges.

Although we are at an early stage of our academic journey, we bring a solid foundation in modern development technologies, a collaborative mindset, and a strong willingness to continuously learn, adapt, and innovate. Our team combines skills across frontend development, backend architecture, database management, and intelligent system design, enabling us to approach problems from both technical and user-experience perspectives.

Through this hackathon, we aim to demonstrate how thoughtful and human-centered technology can significantly improve everyday financial interactions. We believe that many small yet

essential aspects of user experience are often overlooked in payment applications, despite having a major impact on convenience, accessibility, trust, and overall user satisfaction.

Our vision is not only to build functional solutions, but also to refine the subtle details that make digital payment systems more seamless, intuitive, and reliable for users. By focusing on simplifying recurring payment management and enhancing user convenience, we hope to deliver a solution that is practical, scalable, and meaningful in real-world financial ecosystems.

As a team, we are driven by curiosity, innovation, and the belief that impactful technology is created by understanding and improving even the simplest user experiences.

1.2 Team Members & Responsibilities

1.2.1 Sansar Chhetri - Python (FastAPI)

Sansar Chhetri will be responsible for developing backend services and APIs using Python with FastAPI. His primary role will be to design and implement fast, lightweight, and efficient APIs that support key system functionalities. In the proposed solution, he will focus on implementing specific business logic such as payment scheduling and automation rules, ensuring that these processes run accurately and reliably. Additionally, he will be helping bridge communication between the frontend and backend by providing well-structured and easy-to-consume API endpoints. Overall, he acts as a service/API development layer, enabling smooth data flow and efficient system functionality across the solution.

1.2.2 Nirmal Bista - Database (PostgreSQL)

He will be responsible for designing and managing the PostgreSQL database for the system. His role includes creating an optimized database schema for users, payments, schedules, and transactions to ensure efficient and secure data storage. He will handle relationships, indexing, and query optimization for fast data access, while also ensuring data integrity, consistency, and backups. In addition, he will collaborate with backend developers to support smooth API integration. Overall, he will act as the data backbone of the system.

1.2.3 Rajkumar Tiruwa – UI/UX (Figma)

Rajkumar Tiruwa will be responsible for UI/UX design of the system using Figma. He will design the overall look and user flow of the application through wireframes, prototypes, and user journey maps before development begins. His focus will be on making the system simple, intuitive, and user-friendly. He will also collaborate with the frontend developer to ensure accurate implementation of the designs. Overall, he will act as the architect of the user experience for the project.

1.2.4 Paras Adhikari – Frontend Developer (React JS)

Paras Adhikari will be responsible for frontend development using React JS, where he will build the user-facing part of the system. His main role will be to convert Figma designs into a responsive and functional interface. He will develop key components like buttons, forms, and dashboards to ensure smooth user interaction. In addition, he will integrate frontend with backend APIs for proper data flow. Overall, he will act as the face builder of the system, making it interactive and user-friendly.

1.2.5 Abishek Kunwar - Backend (NestJS)

Abishek Kunwar will be responsible for developing the main backend system using NestJS, where he will build the core application logic of the platform. His role will include handling authentication, user management, and core APIs to ensure secure and structured system operations. He will also integrate the PostgreSQL database with the backend to ensure smooth and reliable data flow. In addition, he will focus on system scalability and proper architecture to support future growth. Overall, he will act as the system core controller, managing the backend infrastructure.

1.3 Why Our Team Can Build This Solution

We are a passionate and collaborative team of BSc IT students dedicated to building practical, scalable, and user-centric digital solutions that address real-world challenges. Our strength lies in our ability to learn rapidly, adapt to new technologies, and transform ideas into meaningful applications through effective teamwork and problem-solving.

As a team, we focus on developing solutions that combine technical efficiency with simplicity, usability, and reliability. We believe that technology delivers the greatest impact when it not only functions effectively but also enhances the overall user experience in a seamless and intuitive way.

Our technical stack includes Figma for user-centered interface design, React for dynamic and responsive frontend development, FastAPI and NestJS for scalable backend architecture, and PostgreSQL for secure and efficient data management. Together, these technologies enable us to design and develop a complete full-stack ecosystem capable of delivering reliable and high-performance solutions.

PaySmart reflects our commitment to thoughtful innovation and our belief that even seemingly small improvements can create a meaningful difference in everyday digital experiences. By focusing on convenience, accessibility, and intelligent payment management, we aim to deliver a solution that simplifies recurring financial activities while improving overall user satisfaction and engagement.

2 Problem Understanding

Digital wallets such as e-Sewa have made everyday transactions faster and more convenient than ever before. Paying electricity bills, recharging the internet, sending money, or settling small expenses can now be done within seconds. Despite this convenience, one important part of financial life still feels unnecessarily manual — managing recurring payments.

For most people in Nepal, recurring payments are a regular part of life. Every month, there are electricity bills from NEA, internet payments to providers like Vianet or WorldLink, rent to be transferred, or subscription services to renew. These are not unexpected expenses — they follow a predictable pattern. Yet, each time they are due, users repeat the same steps: open the app, search for the service, enter details, and complete the payment.

While features like scheduling and automation do exist within the app, they are often hidden inside sections like “More” or require multiple steps to set up. Because of this, many users are either unaware of these features or find them too effortful to use. As a result, manual payment becomes the default behavior.

To understand this better, consider a typical working individual in Kathmandu. Their day is filled with commuting, office work, family responsibilities, and social commitments. Somewhere in between, an electricity bill becomes due. There is no reminder inside the app, and unless they actively check, the date passes unnoticed. A few days later, they remember — sometimes after receiving a late notice or facing service inconvenience. The issue here is not unwillingness to pay, but the lack of a system that supports them at the right moment.

Similarly, think about a small shop owner managing daily transactions, supplier payments, and household expenses. With so many responsibilities, remembering exact payment dates for

recurring bills becomes difficult. Even if reminders come via SMS, they are disconnected from the payment flow, requiring extra effort to act on them.

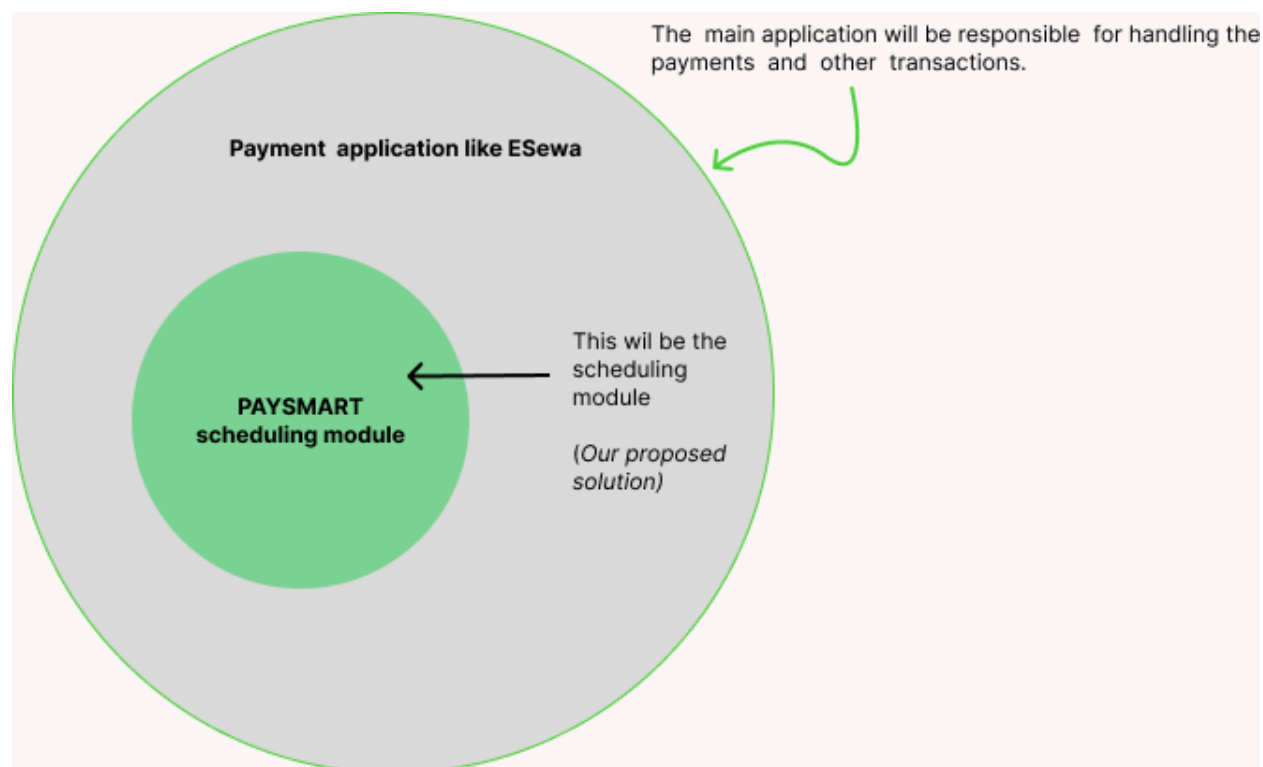
These situations are common across different age groups and professions. Whether it is a salaried employee, a business owner, or someone managing household finances, the pattern is the same — recurring payments require repeated attention despite being predictable.

The key insight is clear. People are not against automation; they actually need it. However, when automation is hidden, complicated, or requires effort to set up, users naturally fall back to manual habits.

Therefore, the real opportunity lies not just in providing automation, but in making it visible, simple, and seamlessly integrated into everyday use — so that managing recurring payments feels effortless rather than something users have to constantly remember.

3 Proposed Solution — PaySmart

Figure: paysmart as an integrated scheduling module



To address the challenges identified in recurring payment management, we propose **PaySmart** — an intelligent payment scheduling and automation module designed to seamlessly integrate into digital wallet platforms such as eSewa.

PaySmart is built on the idea that predictable financial activities should not require repetitive manual effort. Instead of expecting users to navigate complex scheduling features, the system proactively assists them by identifying recurring payment patterns, generating contextual reminders, and enabling quick, low-effort automation directly within the payment flow.

The solution aims to transform digital wallets from simple transactional platforms into proactive financial assistants that understand user behavior and simplify everyday financial management. By making automation more visible, intuitive, and seamlessly integrated, PaySmart helps reduce missed payments, repetitive actions, and user inconvenience.

For recurring expenses such as electricity bills from Nepal Electricity Authority, internet subscriptions, rent, or monthly renewals, PaySmart provides timely suggestions and simplified scheduling options that enhance both convenience and user experience.

3.1 Core features of the proposed solution:

3.1.1 Smart Bill Alerts

One of the core features of PaySmart is the introduction of intelligent bill alerts. Currently, users often receive reminders from service providers such as ISPs or educational institutions through SMS or email on their devices (Basavaraju et al., 2025). However, these reminders remain disconnected from the payment platform, requiring users to manually initiate the payment process.

PaySmart bridges this gap by bringing the entire experience into a single, unified interface. For this during the initial onboarding, users are presented with a simple guided walkthrough that helps them quickly set up their preferences. In this step, users specify the services they regularly pay for, such as internet subscriptions, college fees, or utilities.

This onboarding process acts as a foundational data source for the system. Based on the information provided, PaySmart can connect with relevant service provider APIs and fetch

necessary billing details directly into the application. As a result, users no longer need to manually search for billers or enter details repeatedly, as the system already understands their payment habits and preferences.

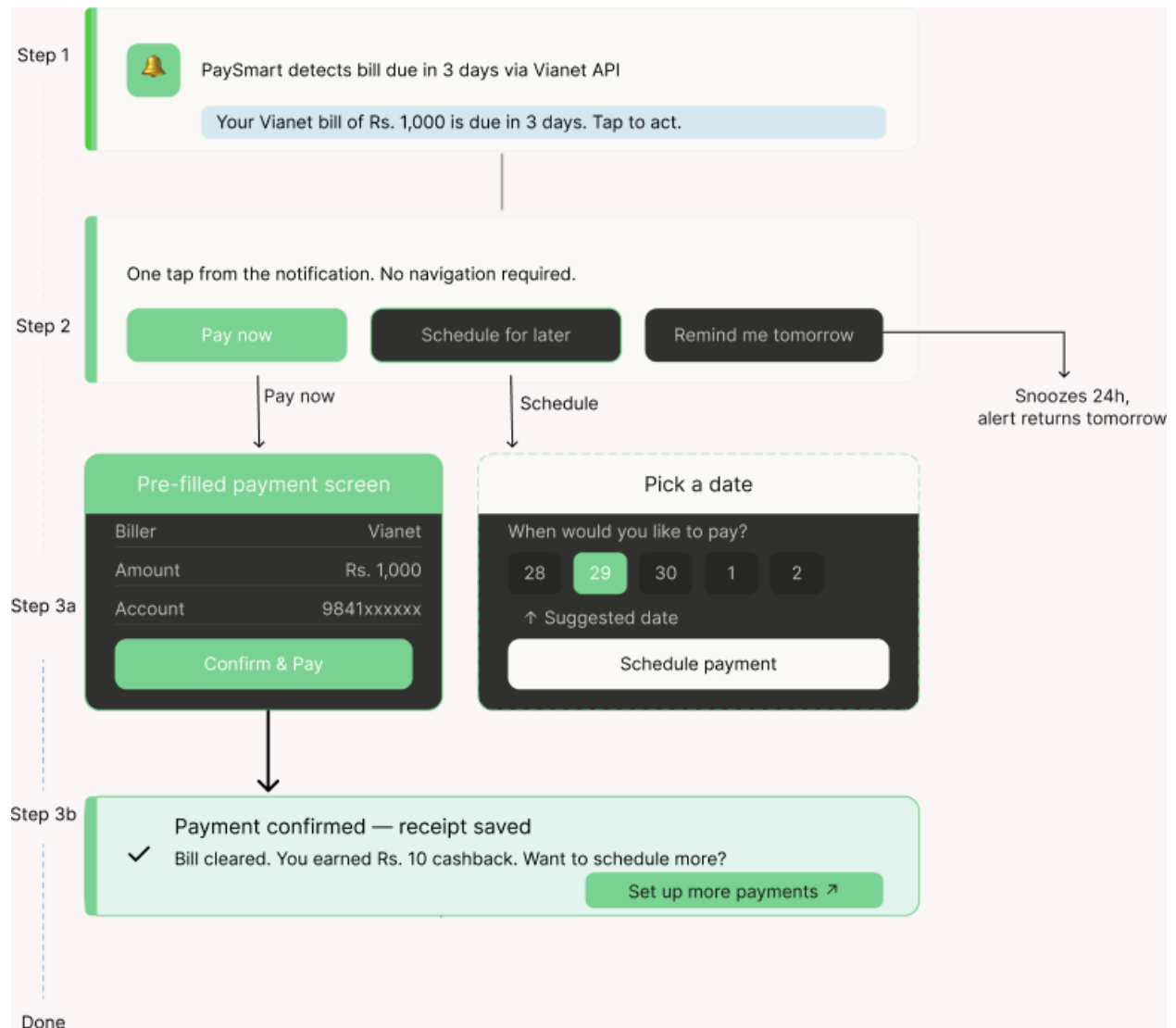
By combining this initial setup with real-time inputs (such as message-based bill detection), PaySmart ensures that relevant payment information is surfaced at the right time in a clear and actionable way.

By combining these two data sources (user-provided preferences from onboarding and contextual information from message). PaySmart is able to accurately identify due payments. Once a bill is detected, the user receives an in-app notification containing all relevant details, including the payee (service provider) and the payable amount.

For example, a user might receive a notification stating that their internet bill of NPR 1,000 is due in three days. Upon tapping the notification, the user is taken directly to a pre-filled payment screen where the service provider, amount, and account details are already populated. The user can complete the payment instantly with a single confirmation.

Alternatively, if the user prefers to pay later, they can schedule the payment directly from the same interface. This eliminates the need to re-enter information or navigate through multiple screens, significantly improving efficiency while maintaining full user control.

Figure: Flow of the bill alert in paysmart



3.1.2 AI-Based Pattern Detection and Smart Suggestions

A defining feature of PaySmart is its ability to learn from user behavior. Instead of requiring users to manually define recurring payments, the system analyzes transaction history to identify patterns.

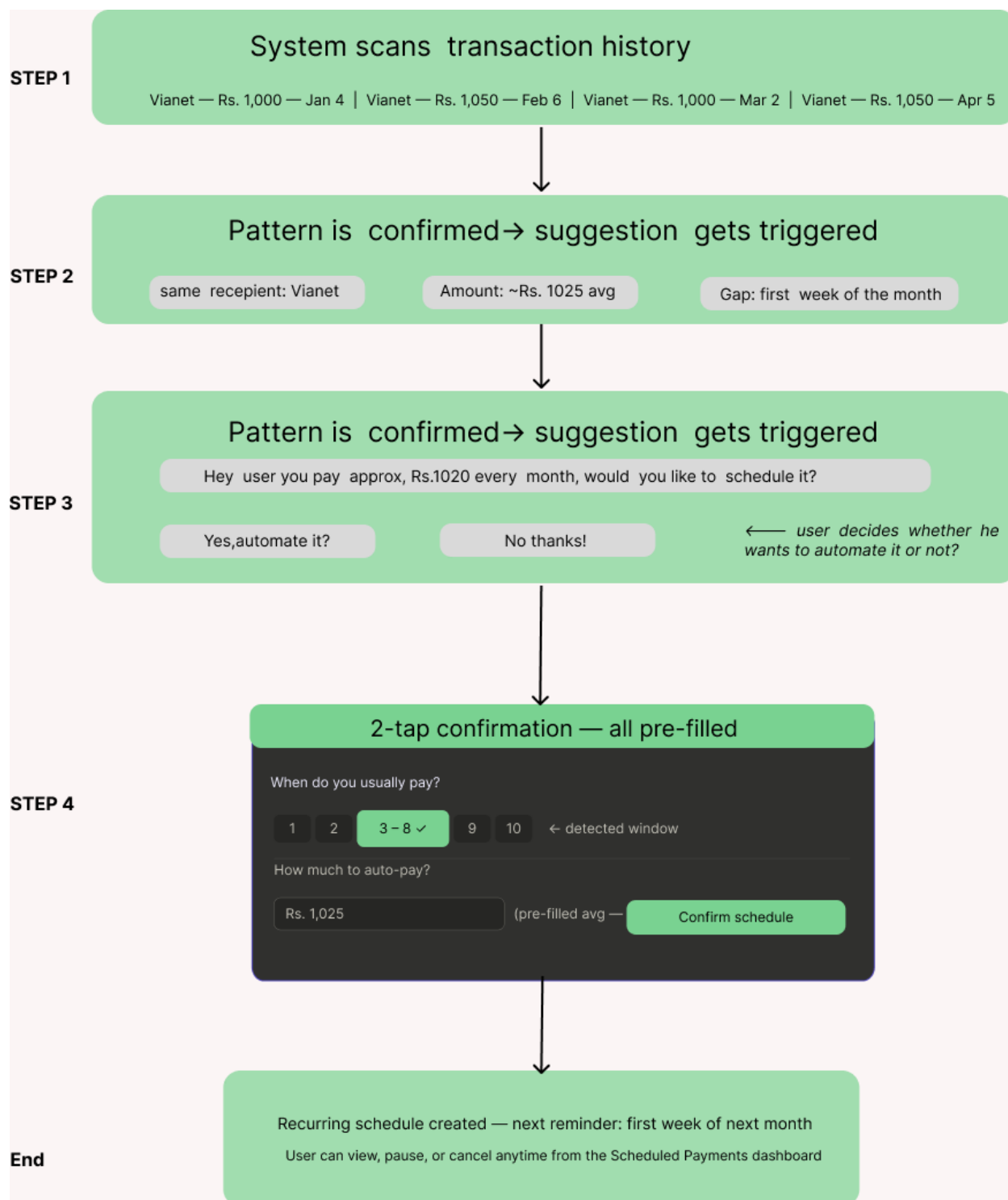
The detection logic is based on three key signals: consistency in the recipient, similarity in transaction amounts, and regularity in time intervals. When these conditions are met, the system identifies a recurring payment pattern and generates a suggestion.

For instance, if a user has paid a particular internet service provider approximately NPR 1,050 every month for the past four months, the system recognizes this as a recurring payment. It then prompts the user with a suggestion to automate this payment.

What makes this approach particularly effective is its adaptability to real-world behavior. Users do not always pay on the exact same date each month. Instead of enforcing rigid scheduling, PaySmart identifies a “payment window.” For example, if payments typically occur between the 1st and 7th of each month, the system uses this range to suggest reminders or automation.

When the user accepts the suggestion, the system presents a simple confirmation interface with pre-filled details. The user only needs to confirm or make minor adjustments, reducing the setup process to just a few interactions.

Figure: smart suggestion for scheduling



3.2 Additional/suitable features for the proposed solution:

3.2.1 Flexible Scheduling System

Traditional scheduling systems often require users to specify exact dates and fixed amounts, which can be restrictive and impractical. PaySmart addresses this limitation by introducing a flexible scheduling mechanism.

The system allows users to define payment windows instead of fixed dates. This flexibility accommodates variations in user income cycles and payment habits. For example, a user who receives their salary at different times each month can still maintain a consistent payment schedule without being constrained by rigid deadlines.

In addition, the system supports dynamic amounts for payments such as electricity bills, where the payable amount varies each month. Instead of enforcing a fixed value, PaySmart updates the scheduled payment amount based on the latest available data, either through SMS parsing or future API integrations.

This flexibility ensures that automation remains accurate and relevant without requiring constant manual adjustments.

3.2.2 Minimal-Setup Manual Scheduling

While automation and suggestions are central to PaySmart, the system also supports manual scheduling for users who prefer direct control. However, unlike traditional systems that rely on complex forms, PaySmart emphasizes simplicity.

The manual scheduling process is designed to be completed in three intuitive steps. The user selects the service provider, confirms a suggested time window, and verifies the payment amount. Since the system pre-fills most of the information based on previous transactions, the user is never required to start from scratch.

This approach significantly reduces cognitive load and ensures that even first-time users can set up schedules effortlessly.

3.2.3 Low Balance Awareness and Preventive Alerts

A common issue with automated payments is the risk of failure due to insufficient balance. PaySmart addresses this proactively by introducing a low-balance alert system.

Before executing a scheduled payment, the system checks the user's wallet balance. If the balance is insufficient, the user is notified in advance and presented with actionable options.

These options may include topping up the wallet, making a partial payment, or rescheduling the transaction.

For example, if a user has a scheduled payment of NPR 1,000 but only has NPR 250 in their wallet, the system informs them a day in advance and allows them to take corrective action. This prevents unexpected failures and improves user trust in the system.

4 In-Scope and Out-of-Scope

The in-scope and out-of-scope areas for the PaySmart solution are outlined as follows:

4.1 In-Scope

- Integration with eSewa Sandbox APIs (Transaction History API + Payment API for automation)
- Integration with Biller / Utility APIs (Internet, Electricity, Water, TV, etc.)
- Transaction synchronization and PostgreSQL storage.
- Pattern Detection Engine — detects same recipient, similar amount ($\pm 15\%$), repeating time window(interval/bound).
- Confidence score calculation and suggestion deduplication (30-day cooldown).
- Recurring payment scheduling and automated execution on due dates.
- WebSocket Gateway for real-time client updates.
- Mobile App (React Native / Flutter) and Web Dashboard (React).
- Six NestJS Core Modules: Auth, Users, Transactions, Notifications, Suggestions, Recurring Payments, and Analytics.

4.1 Out of Scope

- ML-based pattern detection (Python — Future Enhancement only).
- Multi-wallet integration beyond eSewa.

- Advanced fraud detection mechanisms
- Analytics Module (included in architecture but not core MVP deliverable)

The PaySmart solution focuses on enhancing the user experience of recurring payments within the existing e-Sewa infrastructure. It includes intelligent scheduling, reminders, and behavioral analysis to simplify financial management.

However, certain aspects are intentionally excluded from the scope of the current implementation. These include direct bank integrations beyond existing APIs, regulatory changes, and large-scale production-level machine learning pipelines. Additionally, deep integrations with third-party service providers are considered a future enhancement rather than a core requirement for the MVP.

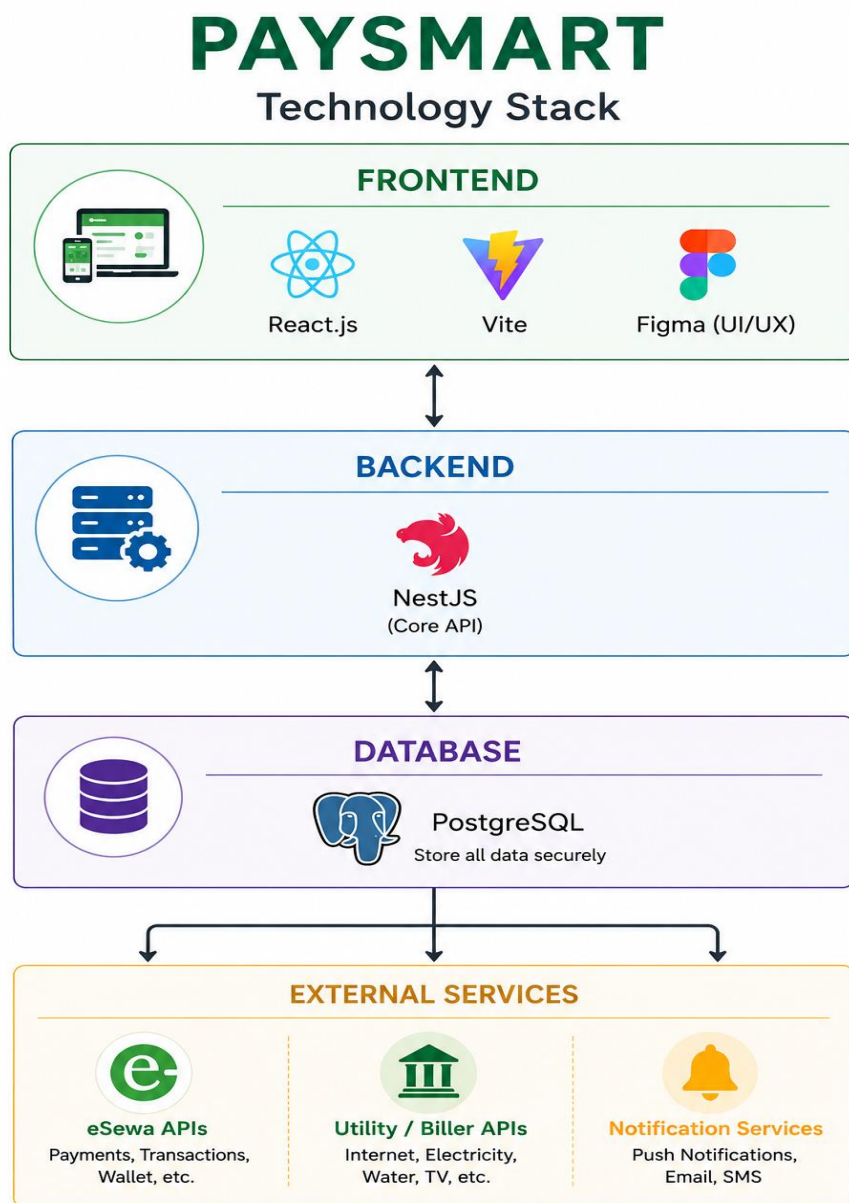
5 User Journey

The user journey within PaySmart is designed to be simple, intelligent, and highly user-centric. The process begins when the system analyzes transaction history and identifies recurring payment behavior or detects an upcoming bill. Based on these patterns, PaySmart proactively generates contextual reminders and smart automation suggestions for the user.

Once the user interacts with the notification, they are guided to a pre-filled payment interface where essential details such as the biller, amount, and payment schedule are already prepared. The user can then choose to make the payment instantly, schedule it for a future date, or enable recurring automated payments with minimal effort.

As users continue interacting with the platform, the system gradually learns from their financial behavior and recurring payment habits. This allows PaySmart to deliver increasingly personalized recommendations, timely reminders, and smarter automation experiences over time. Ultimately, the solution reduces repetitive manual actions and transforms recurring payment management into a more seamless, convenient, and proactive experience.

6 Technology Stack



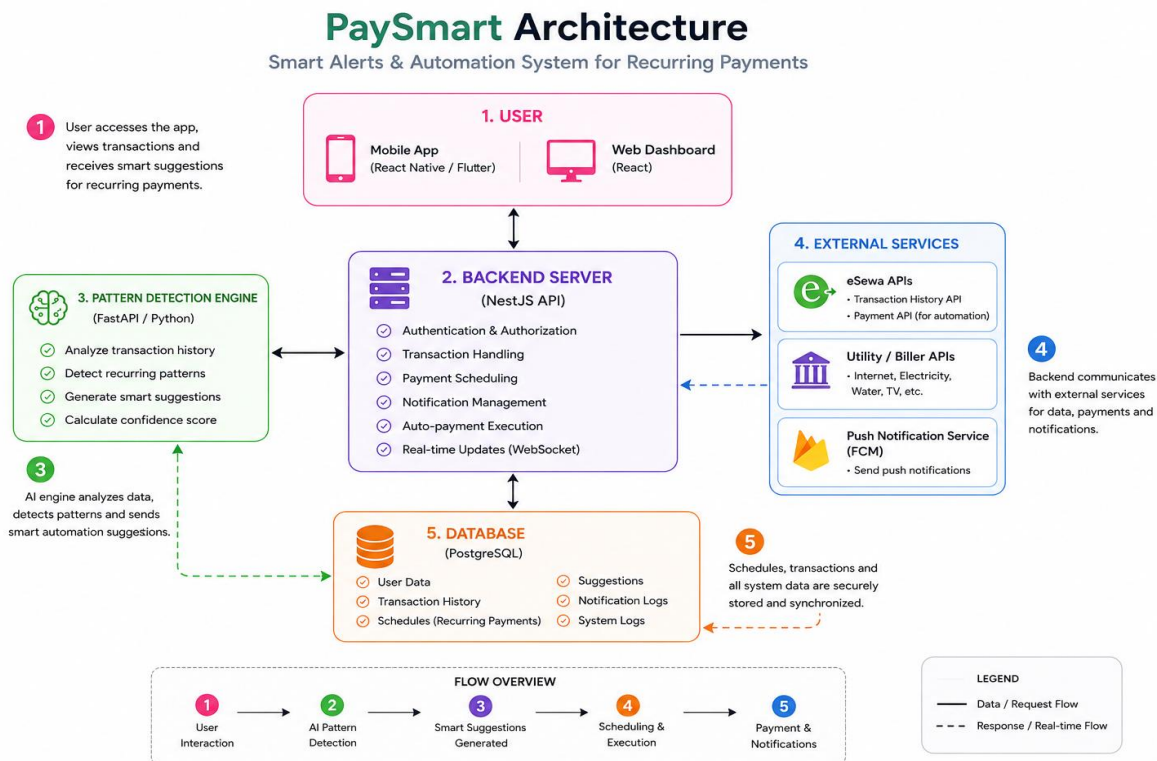
The implementation of PaySmart leverages a modern and scalable technology stack aligned with the hackathon requirements.

The frontend is built using React.js with TypeScript, ensuring a responsive and maintainable user interface. UI/UX designs are created using Figma to deliver a clean and intuitive experience.

The backend is powered by Node.js using the NestJS framework, which provides a structured and scalable architecture for handling APIs, scheduling logic, and integrations. We are also proposing AI-based pattern detection, a Python-based microservice is implemented using FastAPI. This service is responsible for analyzing transaction data and generating intelligent suggestions.

PostgreSQL is used as the primary database for storing user schedules, transaction metadata, and system logs. The entire system is version-controlled using GitHub, ensuring collaborative development and code quality.

7 Architecture Overview



The PaySmart system follows a modular and scalable architecture designed to ensure efficient communication between all system components. As illustrated in the diagram, users interact with the platform through the mobile app or web dashboard, while the frontend securely communicates with the backend through APIs.

The backend, developed using NestJS, serves as the core system layer responsible for authentication, transaction handling, payment scheduling, notification management, and automated payment execution. It also manages communication with external services such as eSewa APIs and utility providers.

To support intelligent automation, PaySmart includes a separate AI-based Pattern Detection Engine built with FastAPI. This service analyzes transaction history, detects recurring payment patterns, and generates smart scheduling suggestions. All system data, including transactions, schedules, and suggestions, are securely managed using PostgreSQL.

This separation of concerns ensures that the system remains flexible, maintainable, and scalable while allowing future enhancements and intelligent features to be integrated efficiently.

8 Security & Privacy

Security and user trust are fundamental aspects of the PaySmart system. The solution is designed to ensure secure handling of payment operations, recurring scheduling, and user data throughout the platform. All communications between the frontend, backend, and external services are conducted through authenticated and secure API channels to minimize unauthorized access risks.

User transaction records, scheduling information, and system data are securely managed within the PostgreSQL database using controlled access mechanisms and proper validation techniques. In addition, the system follows a confirmation-based automation approach, ensuring that recurring payments are never activated without user approval.

To maintain transparency and user confidence, PaySmart also provides real-time notifications and transaction updates for scheduled and automated payments. These security-focused practices help ensure reliability, privacy, and safe financial interactions within the eSewa ecosystem.

9 Expected Impact

PaySmart has the potential to significantly improve both user experience and platform engagement. For users, it reduces the likelihood of missed payments, simplifies financial management, and saves time. For e-Sewa, it increases transaction frequency, enhances user retention, and strengthens its position as a user-centric fintech platform.

10 Intelligent Payment Scheduling & Automation (Overall Working Mechanism)

PaySmart is built as an intelligent, event-driven system that automates recurring payments by continuously learning from user transactions and executing payments through a secure, rule-based workflow within the eSewa ecosystem.

10.1 Transaction Data Synchronization

The system begins by securely connecting the user's eSewa account and synchronizing transaction history via eSewa Sandbox APIs. All transaction records are structured and stored in PostgreSQL for real-time and historical analysis.

10.2 Behavioral Pattern Analysis

A scheduled backend engine (NestJS Cron Jobs) continuously analyzes transactions to detect recurring behavior. It evaluates three key parameters:

- Consistent recipient (payee matching)
- Stable transaction amount ($\pm 15\%$ variation)
- Repeating time intervals

These signals are combined to generate a confidence score for identifying recurring payments.

10.3 Intelligent Suggestion Engine

When a recurring pattern is confirmed, the system generates an automation suggestion. To ensure relevance and avoid repetition, a cooldown mechanism (30 days) is applied. Suggestions are instantly delivered through WebSocket and Firebase Cloud Messaging (FCM).

10.4 User Confirmation & Auto-Setup

Users can activate automation through a one-tap approval flow. The system automatically pre-fills payment details, schedules, and recipient information to ensure a frictionless setup experience.

10.5 Scheduling & Pre-Execution Validation

Approved payments are registered into the recurring payment scheduler. Before execution, the system performs safety checks:

- Wallet balance verification
- Bill availability confirmation
- Transaction status validation

This ensures accuracy and prevents failed payments.

10.6 Automated Payment Execution

On the scheduled date, payments are automatically executed via the eSewa Payment API. The system ensures secure, reliable, and timely transaction processing.

10.7 Real-Time Notification System

Throughout the entire process, users receive live updates, alerts, and confirmations via WebSocket and FCM, ensuring full transparency and control.

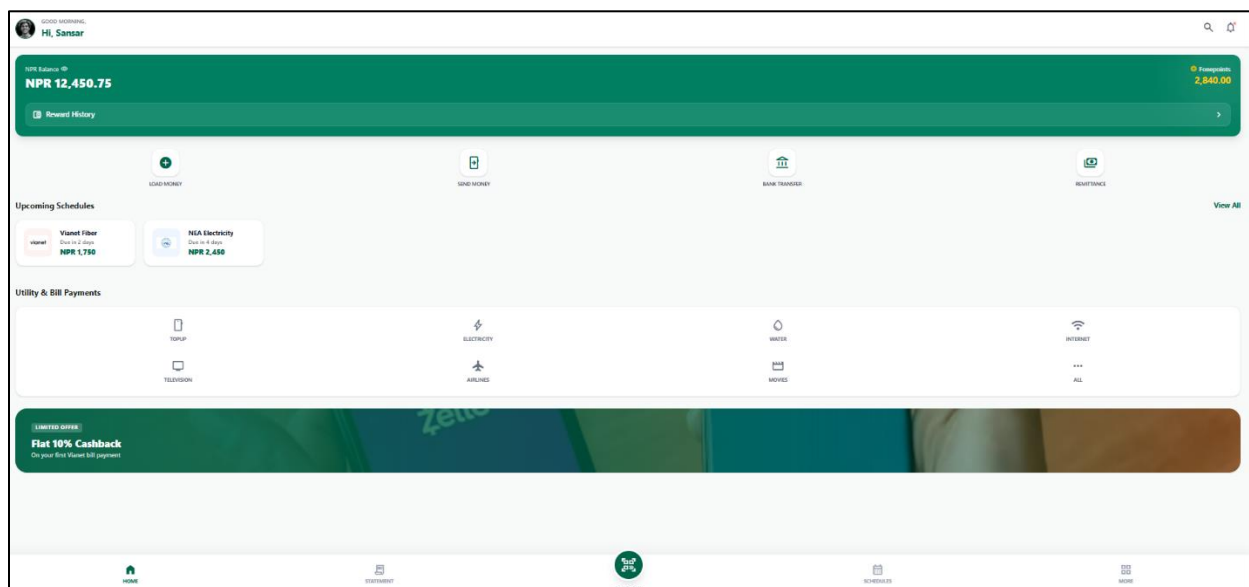
10.8 End-to-End Flow

The end to end workflow of the proposed system is shown in the given image:

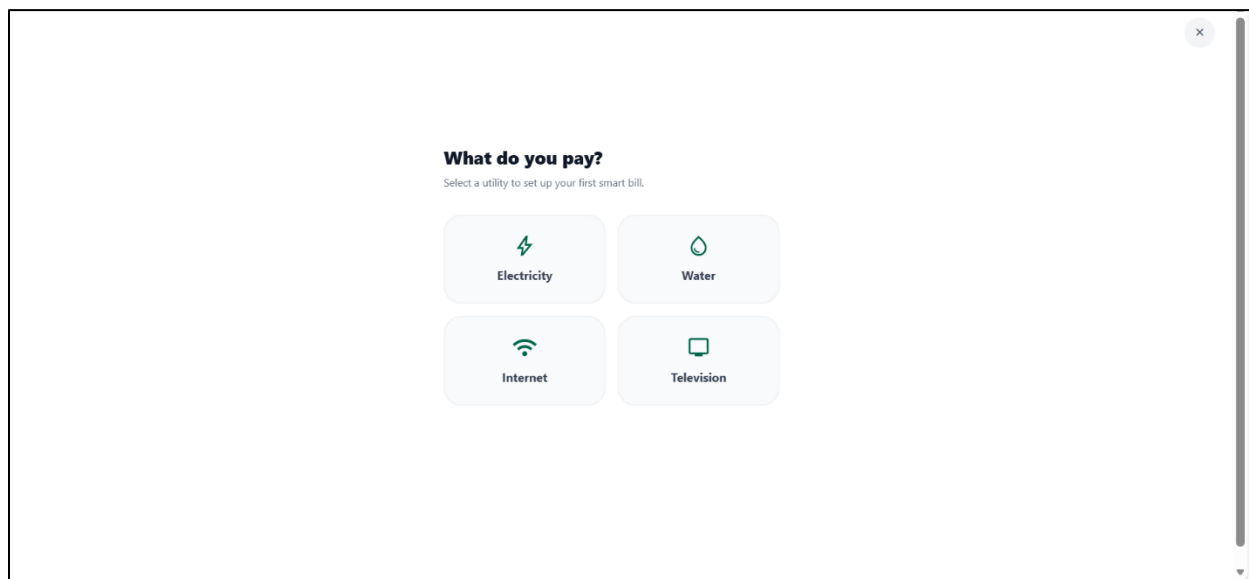
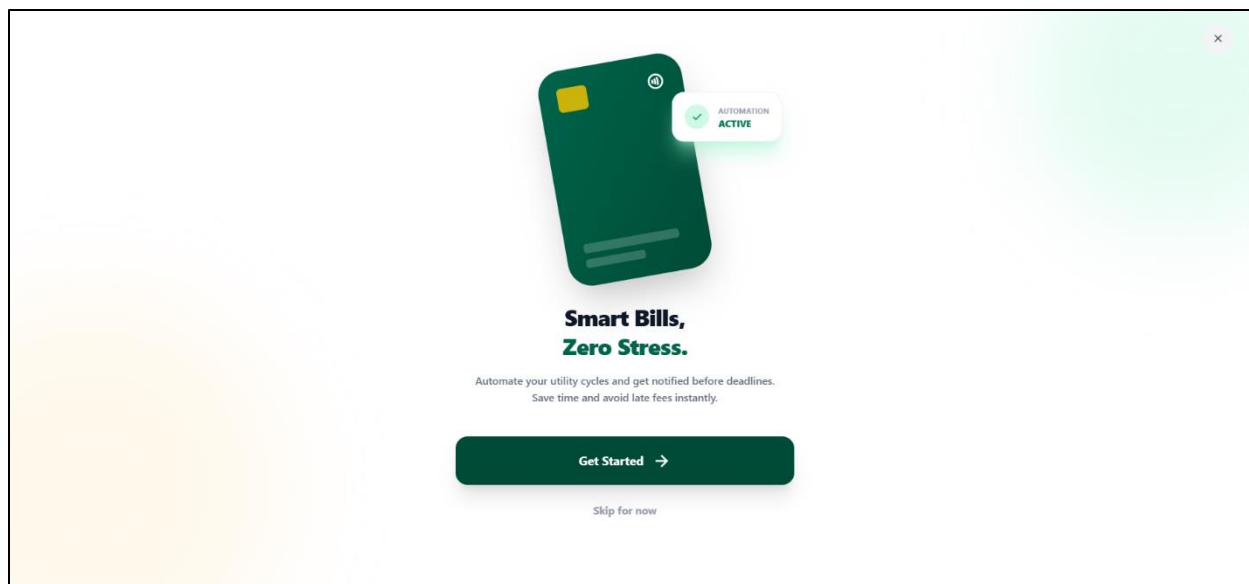


11 Sample User Interfaces of the proposed solution

UI of the dashboard



User interface of Walkthrough





Add Internet details?

Adding details now lets us track your bills and notify you automatically.

Yes, add details

No, do it later

Internet Account

Enter your customer ID or account number.

CUSTOMER ID / SC NO.

101|

Save and Complete

UI of smart Suggestions

Smart Suggestions

AI INSIGHT

Hey, you usually pay Vianet around this time!

Would you like to automate this to avoid any service interruptions? We can handle it for you.

Automate Now

Dismiss

HOME

STATEMENT

Scan & Pay

SCHEDULES

MORE

Auto-pay Setup

Vianet Internet

Subscription Bill

MAXIMUM AMOUNT (NPR)

1000

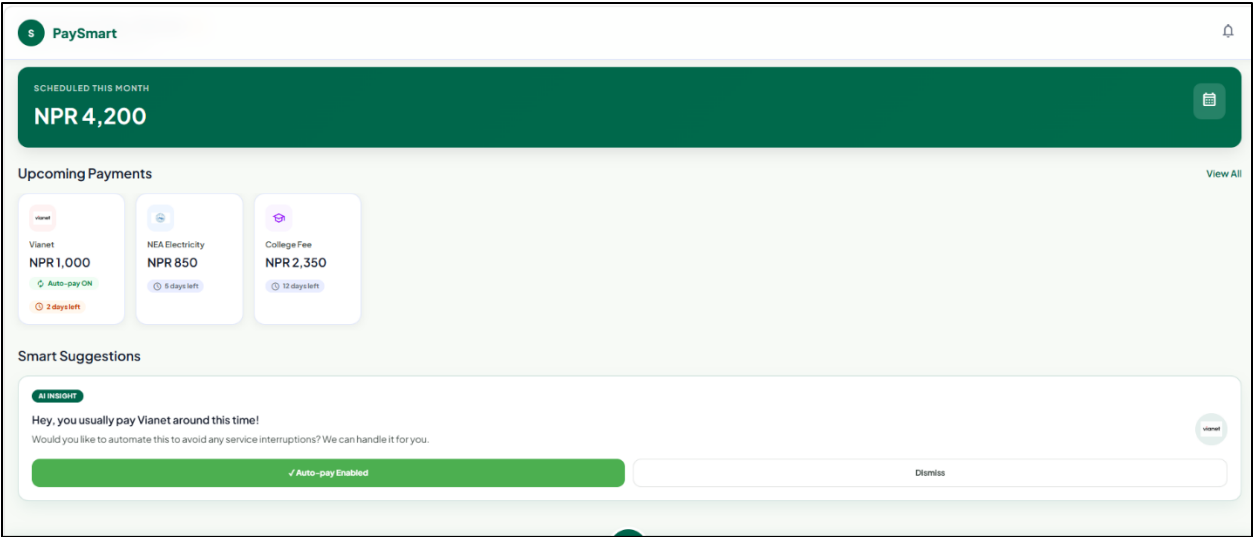
PREFERRED DEDUCTION DATE

05/15/2026

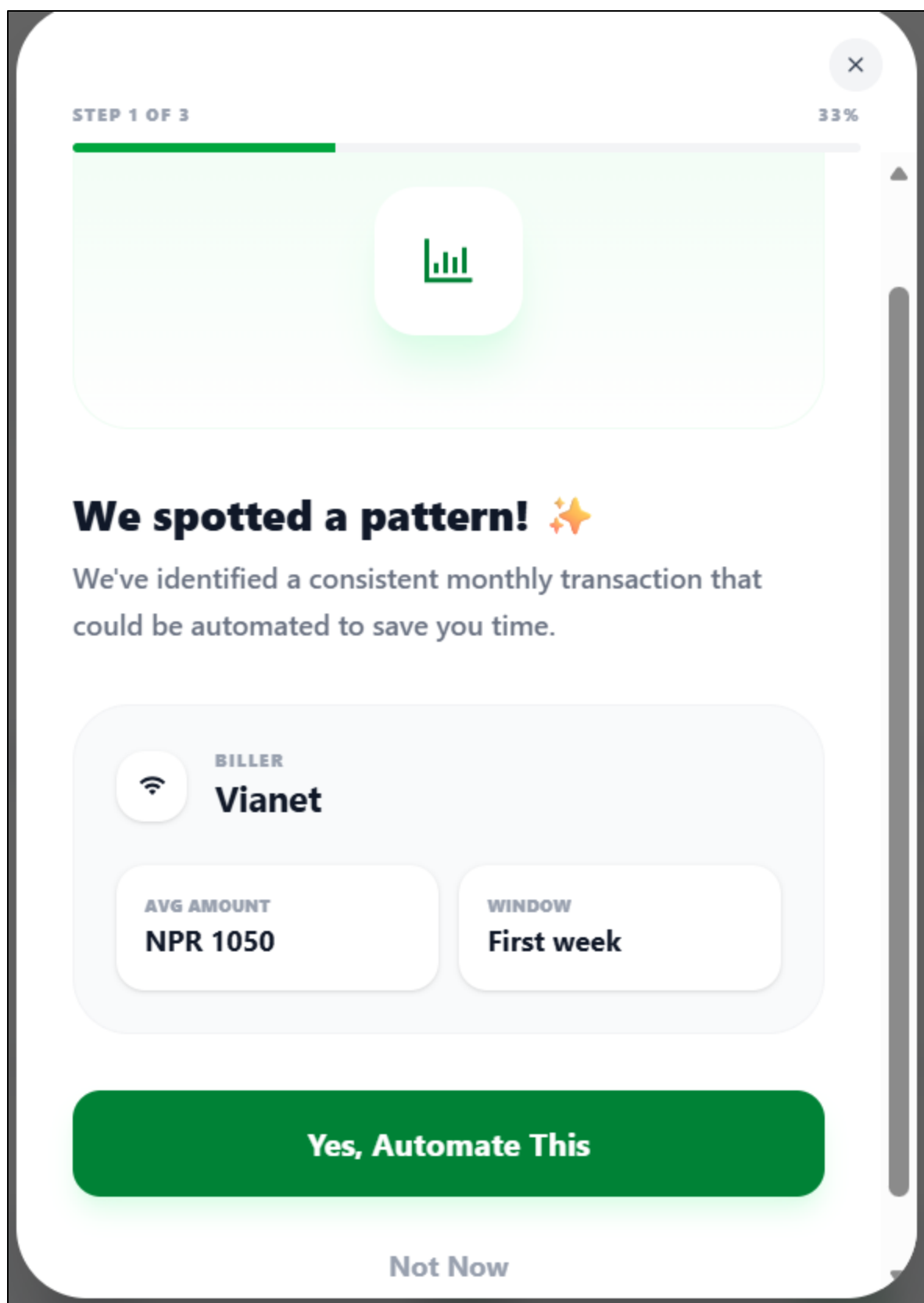
Reminder Notification

Send alert 1 day before deduction

Confirm & Enable



Or



×

STEP 2 OF 367%

When do you pay?
We've highlighted your usual window.

OCTOBER 2023<>

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

Looks good, Next

×

Set your auto-pay limit

You can change this anytime later

MONTHLY LIMIT

NPR **1,050** 

Scheduled Date **5th of month**

Provider **Vianet**

Notify Me ☐

Auto-Deduct ☐

Confirm Schedule

12 Conclusion

PaySmart transforms recurring digital payments into a smart, proactive, and user-friendly experience. By combining intelligent bill alerts, AI-based payment pattern detection, flexible scheduling, and low-balance notifications, the system reduces manual effort and helps users manage payments more efficiently. Built using scalable technologies such as React.js, NestJS, FastAPI, and PostgreSQL, PaySmart is designed for reliability, maintainability, and future growth. The solution enhances user convenience while increasing engagement and recurring transaction activity for eSewa. Overall, PaySmart demonstrates how intelligent automation and user-centered design can simplify everyday financial management and improve the digital payment experience in Nepal.

Reference

Basavaraju, K. V., Narahari, L., Uppara, H., Paruchuru, M. K., Chimbili, G. P., & Dudekula, B. F. (2025). EduPay : College Fee Management Portal. *Global Journal of Engineering Innovations and Interdisciplinary Research*, 5(2). <https://doi.org/10.33425/3066-1226.1086>